Collective Communication in MPI
Lab 3

Adi Ramanarayan
230905380 48
A2

Lab Exercises:

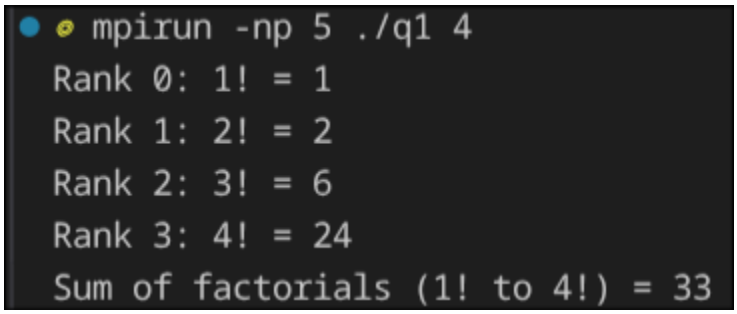
Q1. Program to find sum of factorials using scan and collective communication.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

long long factorial(int n) {
    long long fact = 1;
    for (int i = 2; i <= n; i++)
        fact *= i;
    return fact;
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int n;
```

```
    if (rank == 0) {
        if (argc < 2) {
            fprintf(stderr, "Usage: mpirun -np <p> %s <n>\n",
argv[0]);
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
        n = atoi(argv[1]);
    }

    MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);

    long long local_sum = (rank < n) ? factorial(rank + 1) : 0;

    long long scan_sum;
    MPI_Scan(&local_sum, &scan_sum, 1, MPI_LONG_LONG_INT, MPI_SUM,
MPI_COMM_WORLD);

    if (rank < n) {
        printf("Rank %d: %d! = %lld\n", rank, rank + 1, local_sum);
    }

    if (rank == n - 1) {
        printf("Sum of factorials (1! to %d!) = %lld\n", n,
scan_sum);
    }

    MPI_Finalize();
    return 0;
}
```

Output:



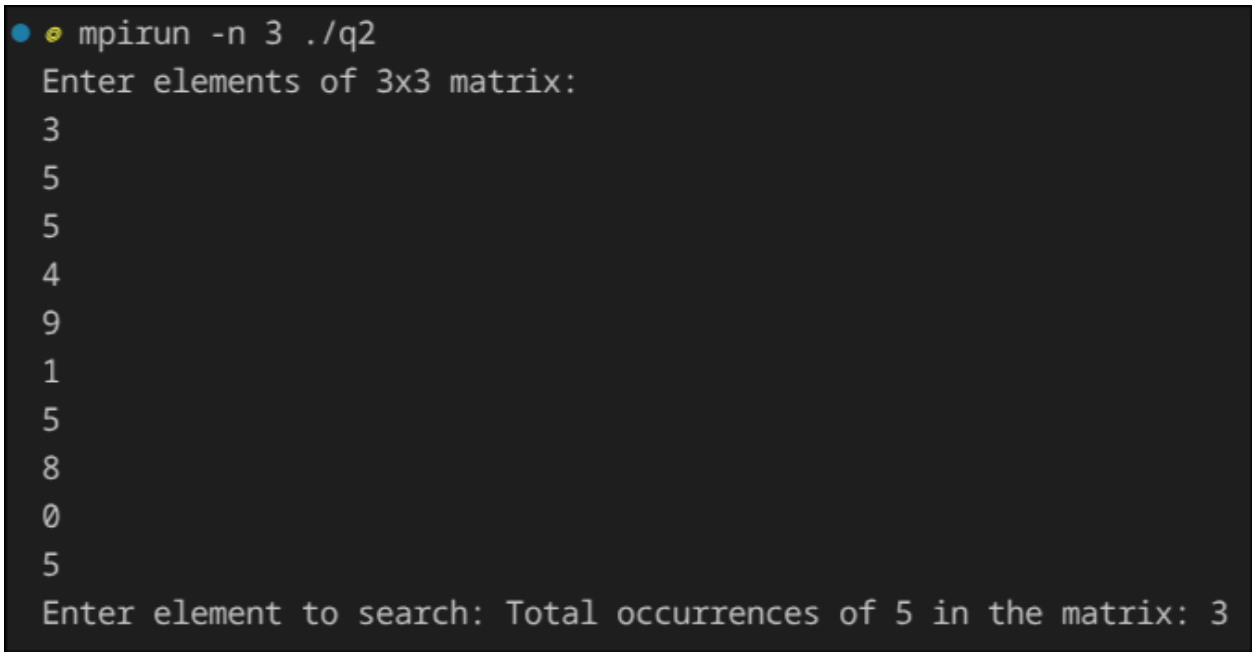
Q2. The program reads a 3X3 matrix. Element is entered by the user to be searched, in root. Count of total occurrences of the element has to be found, using 3 processes.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char** argv) {
   MPI_Init(&argc, &argv);

   int rank, size;
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   if (size != 3) {
       if (rank == 0) {
           fprintf(stderr, "This program requires exactly 3
processes.\n");
       }
       MPI_Abort(MPI_COMM_WORLD, 1);
   }

   int matrix[3][3];
   int element_to_search;
   int local_count = 0;

   if (rank == 0) {
       printf("Enter elements of 3x3 matrix:\n");
       for (int i = 0; i < 3; i++) {
           for (int j = 0; j < 3; j++) {
               scanf("%d", &matrix[i][j]);
           }
       }
       printf("Enter element to search: ");
       scanf("%d", &element_to_search);
   }

   MPI_Bcast(&element_to_search, 1, MPI_INT, 0, MPI_COMM_WORLD);
   MPI_Scatter(matrix, 3, MPI_INT, matrix[rank], 3, MPI_INT, 0,
MPI_COMM_WORLD);

   for (int j = 0; j < 3; j++) {
```

```
        if (matrix[rank][j] == element_to_search) {
            local_count++;
        }
    }

    int total_count;
    MPI_Reduce(&local_count, &total_count, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Total occurrences of %d in the matrix: %d\n",
element_to_search, total_count);
    }

    MPI_Finalize();
    return 0;
}
```

Output:



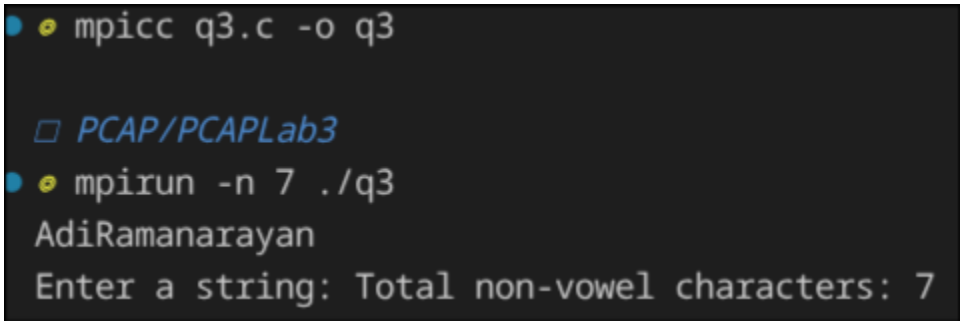
Q3. The program reads a string. Uses N processes to divide the string and count the number of non vowel characters.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mpi.h>
#include <ctype.h>

int is_vowel(char c) {
    c = tolower(c);
    return (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c ==
'u');
}
int main(int argc, char** argv) {
    int rank, size;
    char *str = NULL;
    int total_non_vowels = 0;

    if (MPI_Init(&argc, &argv) != MPI_SUCCESS) {
        fprintf(stderr, "Error initializing MPI\n");
        return EXIT_FAILURE;
    }
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        printf("Enter a string: ");
        str = (char *)malloc(100 * sizeof(char)); // Initial
allocation
        if (fgets(str, 100, stdin) == NULL) {
            fprintf(stderr, "Error reading input\n");
            MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
        }
        str[strcspn(str, "\n")] = 0; // Remove newline character
    }

    int str_length;
    if (rank == 0) {
        str_length = strlen(str);
    }
    MPI_Bcast(&str_length, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

```
    int chunk_size = str_length / size;
    char *local_str = (char *)malloc((chunk_size + 1) *
sizeof(char));

    MPI_Scatter(str, chunk_size, MPI_CHAR, local_str, chunk_size,
MPI_CHAR, 0, MPI_COMM_WORLD);

    int local_non_vowels = 0;
    for (int i = 0; i < chunk_size; i++) {
        if (!is_vowel(local_str[i]) && isalpha(local_str[i])) {
            local_non_vowels++;
        }
    }

    MPI_Reduce(&local_non_vowels, &total_non_vowels, 1, MPI_INT,
MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Total non-vowel characters: %d\n", total_non_vowels);
    }

    free(local_str);
    if (str) free(str);
    if (MPI_Finalize() != MPI_SUCCESS) {
        fprintf(stderr, "Error finalizing MPI\n");
        return EXIT_FAILURE;
    }
    return 0;
}
```

Output:



Q4. Takes 2 input strings, divides them and combines in alternating character fashion.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mpi.h>

#define MAX 100

int main(int argc, char *argv[]) {
    int rank, size, len, i;
    char str1[MAX], str2[MAX];
    char *local_result, *final_result;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        fprintf(stdout, "Enter first string: ");
        fflush(stdout);
        scanf("%s", str1);

        fprintf(stdout, "Enter second string: ");
        fflush(stdout);
        scanf("%s", str2);

        len = strlen(str1);
        if (len != strlen(str2)) {
            fprintf(stderr, "Strings must be of same length\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
    }

    MPI_Bcast(&len, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(str1, len + 1, MPI_CHAR, 0, MPI_COMM_WORLD);
    MPI_Bcast(str2, len + 1, MPI_CHAR, 0, MPI_COMM_WORLD);

    local_result = calloc(2 * len + 1, sizeof(char));

    for (i = rank; i < len; i += size) {
        local_result[2 * i]     = str1[i];
```

```
        local_result[2 * i + 1] = str2[i];
    }

    if (rank == 0)
        final_result = calloc(2 * len + 1, sizeof(char));

    MPI_Reduce(local_result, final_result,
               2 * len + 1, MPI_CHAR,
               MPI_MAX, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Resultant string: %s\n", final_result);
        free(final_result);
    }

    free(local_result);
    MPI_Finalize();
    return 0;
}
```

Output: